APPENDIX E – EXPLAINABILITY GRAPHS - FE C#

E.1 Feature Importance

The standardized features, used both for the detection of Feature Envy (FE) in C# projects and for the assessment of its severity, were selected using the Chi-Square technique (3), considering 30% of the available features.

Figura 25 shows the importance of features for detecting FE in C# projects. To compute feature importance, we used the *LossFunctionChange* option in the *get_feature_importance* method from CatBoost. For each feature, the value represents the difference in model loss with and without that feature. The most prominent features for FE detection in C# projects were `AMW_type` and `WOC_type`.

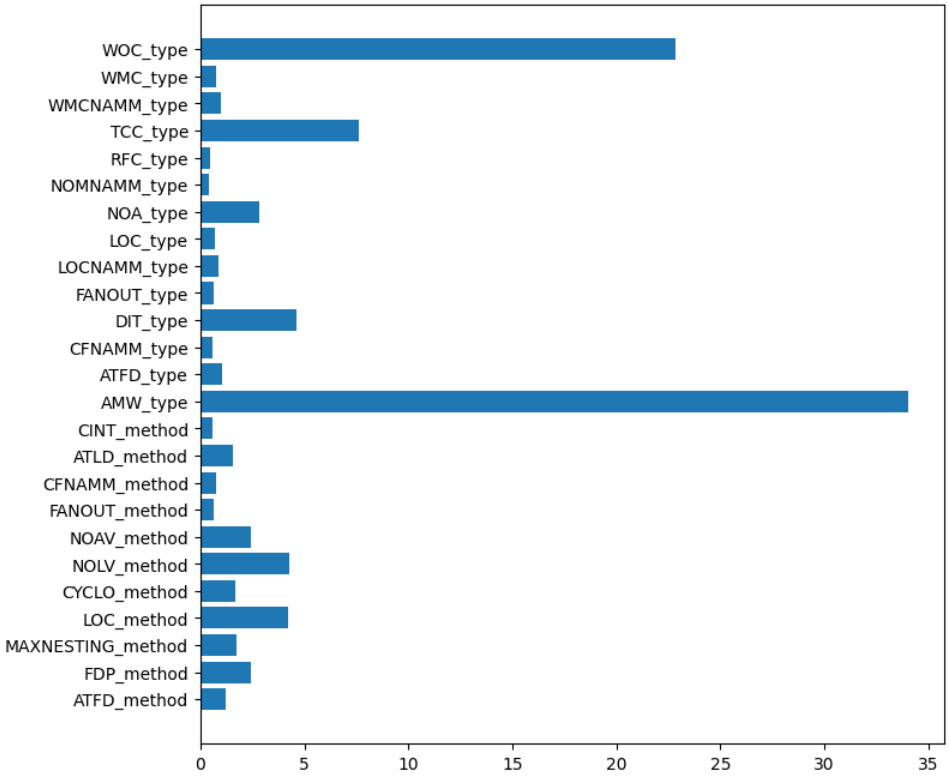

Figura 25 – Feature Importance for FE Detection in C#

Figura 26 shows the importance of features for assessing the severity of FE in C# projects. For this analysis, we used the default *importance_type* parameter in XGBoost, i.e., "*weight*", which measures importance by the number of times a feature appears in the decision tree nodes. The most important features for severity assessment of FE in C# projects were `ATFD_method`, `CYCLO_method`, `NOAV_method`, and `ATFD_type`.

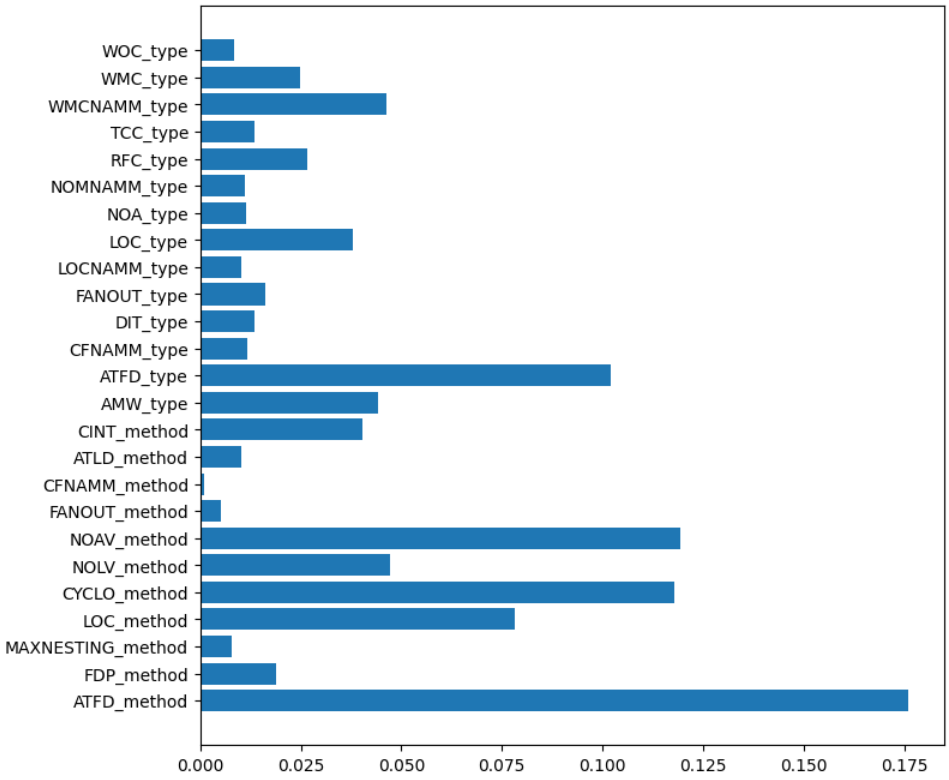

Figura 26 – Feature Importance for Severity Assessment of FE in C#

E.2 LIME

For the severity assessment of FE in C# projects, we used a model based on XGBoost with standardized data. Figura 27 presents the LIME graph for a non-severe FE instance, which achieved 100% prediction probability by the severity assessment model. This instance has several feature values that contributed to its high prediction probability, such as: $-0.05 < \texttt{TCC\_type} \leq -0.02$, $\texttt{AMW\_type} \leq -0.08$, $-0.39 < \texttt{CINT\_method} \leq 0.62$, $-0.38 < \texttt{ATFD\_method} \leq 0.54$, $-0.17 < \texttt{RFC\_type} \leq 0.46$, and $\texttt{NOLV\_method} \leq -0.38$.

In the case of a severe FE instance (see Figura 28), the model achieved a prediction probability of 62%. The feature values that contributed positively to this prediction were: $\texttt{ATFD\_method} > 0.54$, $\texttt{CFNAMM\_type} > 0.71$, $\texttt{WOC\_type} > -0.16$, and $\texttt{AMW\_type} \leq -0.08$. However, the following feature values reduced this prediction probability: $\texttt{NOLV\_method} > 0.88$, $-0.03 < \texttt{LOCNAMM\_type} \leq 0.59$, $-0.39 < \texttt{CINT\_method} \leq 0.62$, and $\texttt{NOA\_type} > 0.30$. Additionally, for this instance, there was a considerable 33% probability of being misclassified as non-severe FE.

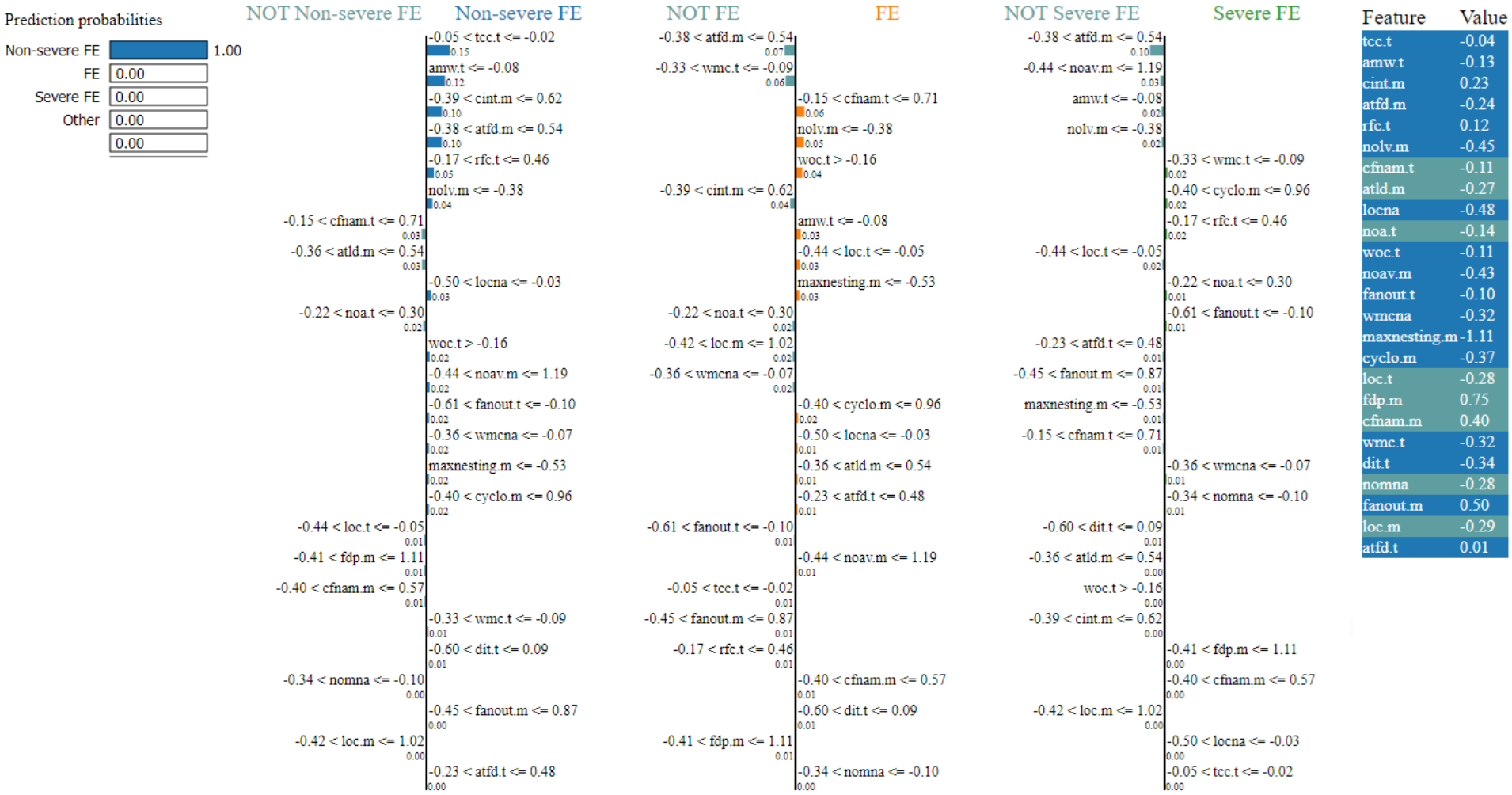

Figura 27 – Explanation of a Non-Severe FE Instance in C# using LIME

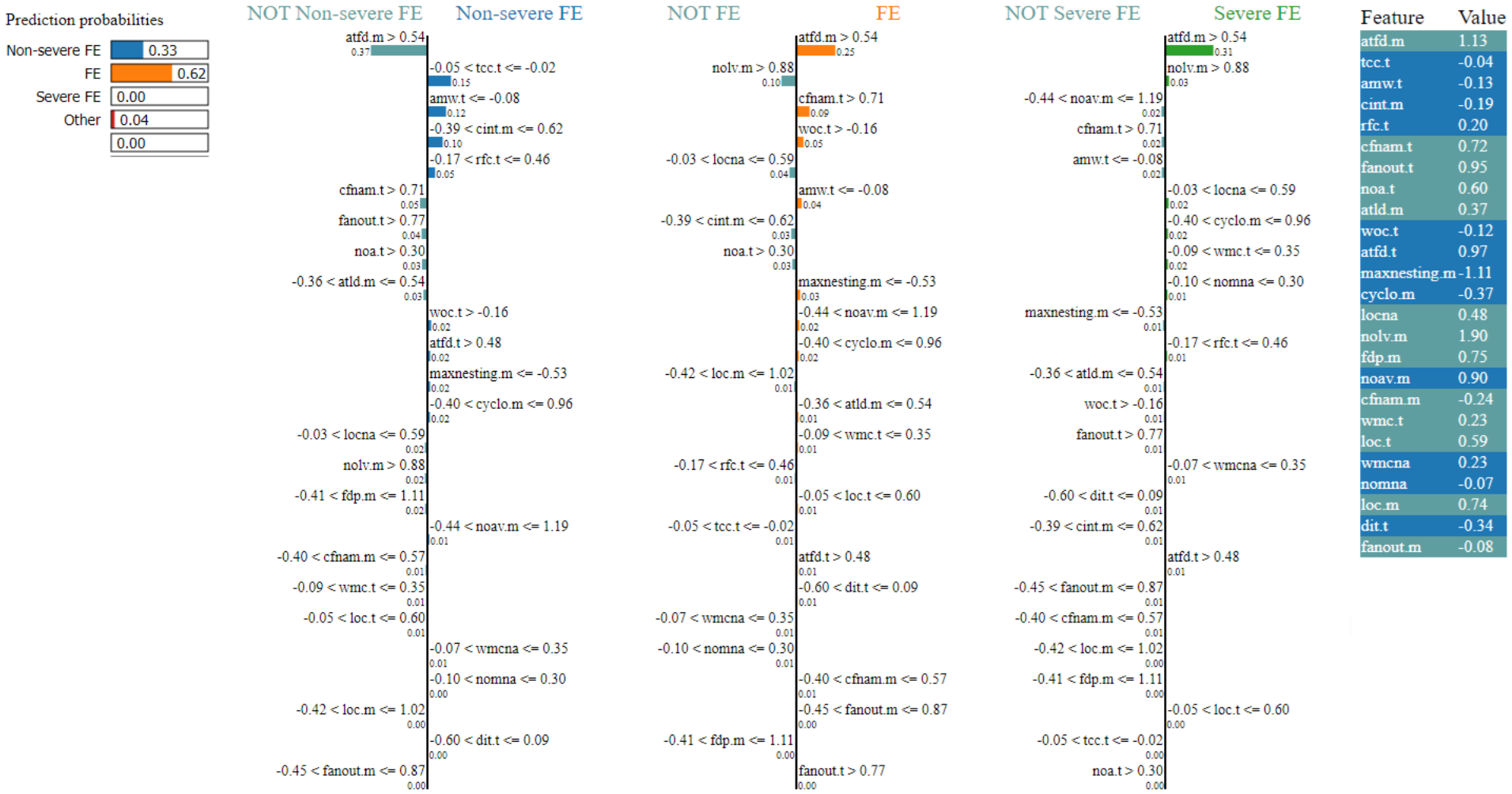

Figura 28 – Explanation of a Severe FE Instance in C# using LIME

APPENDIX F – EXPLAINABILITY GRAPHS - LM C#

F.1 Feature Importance

The standardized features for detecting LM in C# projects were selected using the Chi-Squared technique (3), representing 30% of the available features. Figura 29 shows the feature importance for LM detection in C# projects. The feature importance was computed using the *LossFunctionChange* method from CatBoost's *get_feature_importance*. The most important features for this detection were LOC_method and CYCLO_method.

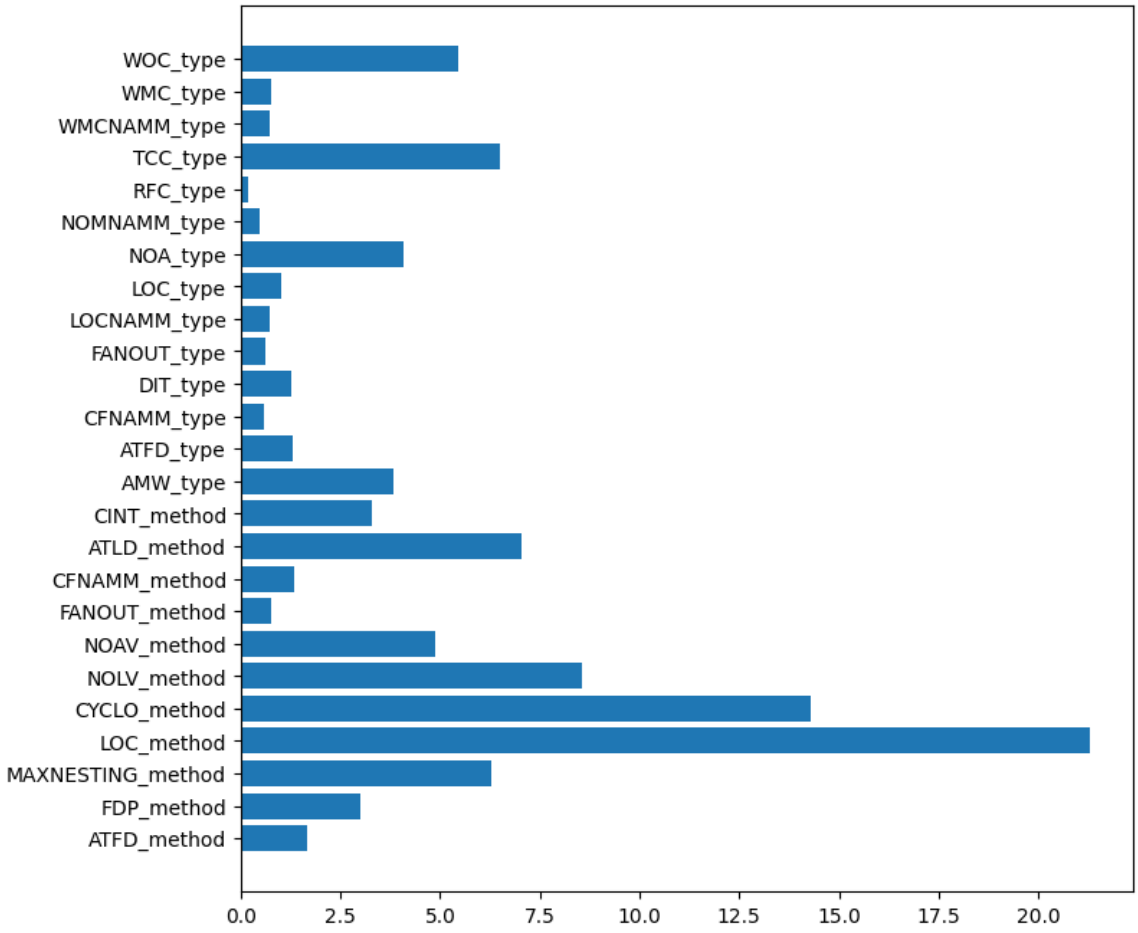

Figura 29 – Feature Importance for LM Detection in C#

Figura 30 shows the feature importance for severity assessment of LM in C# projects. The selected features, corresponding to 25% of the total, were chosen using the ANOVA technique. The importance was calculated based on the total impurity reduction using the *Log Loss* criterion across the model's decision trees. CYCLO_method, LOC_method, NOAV_method, and RFC_method were the most important features for the multiclass severity assessment model in C# projects.

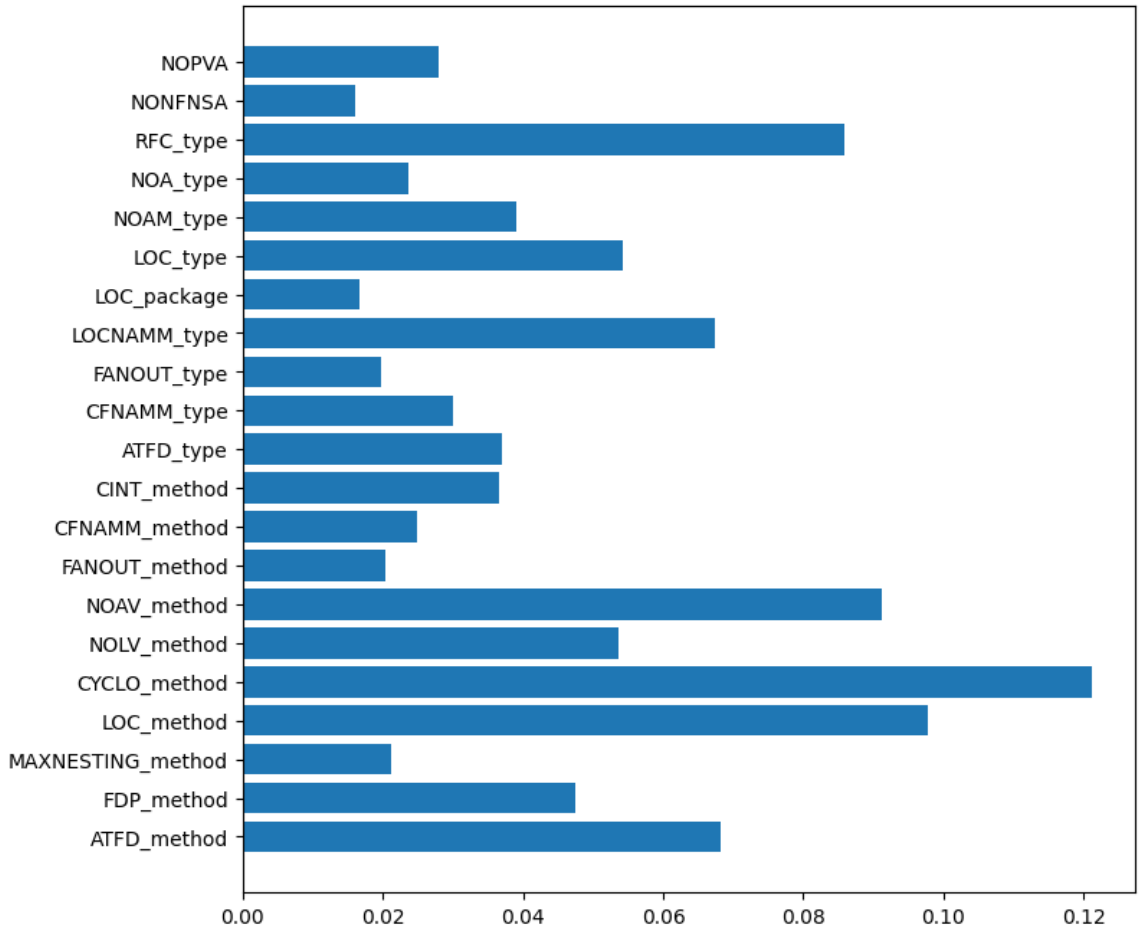

Figura 30 – Feature Importance for LM Severity Assessment in C#

F.2 LIME

For severity assessment of LM in C# projects, a Random Forest-based model was used without the need for data scaling. Figura 31 presents the LIME plot for a non-severe LM instance, which achieved an 80% prediction probability by the severity assessment model. This instance showed several important feature values, such as: $RFC\_type \leq 0.00$, $3.00 < LOC\_method \leq 37.00$, $1.00 < CYCLO\_method \leq 6.00$, $ATFD\_method \leq 0.00$, and $1.00 < NOAV\_method \leq 8.00$. Additionally, there was an 18% probability that the instance would be classified as an LM severity case.

For the LM severity instance shown in Figura 32, the model achieved a prediction probability of 59%. The following feature values positively contributed to this correct prediction: $CYCLO\_method > 13.00$, $LOC\_method > 83.00$, $RFC\_type \leq 0.00$, $NOLV\_method > 12.00$, $ATFD\_method \leq 0.00$, $NOAV\_method > 19.50$, and $CFNAMM\_method > 8.00$. However, some of these attributes, such as CYCLO_method and RFC_type, within the same value ranges, also contributed to considerable prediction probabilities for the other two severity levels: non-severe LM (16%) and severe LM (21%).

Figura 33 details the LIME plot for a severe LM instance, which reached a correct

prediction probability of 77%. The values $CYCLO\_method > 13.00$, $LOC\_method > 83.00$, $NOLV\_method > 12.00$, $CINT\_method > 11.00$, and $FDP\_method > 5.00$ were the main contributors to this correct prediction. Furthermore, it is worth noting that this instance also had a 17% probability of being classified as LM severity, since it shared similar values for CYCLO_method, LOC_method, and NOLV_method with the severe LM class, but with lower contribution weights.

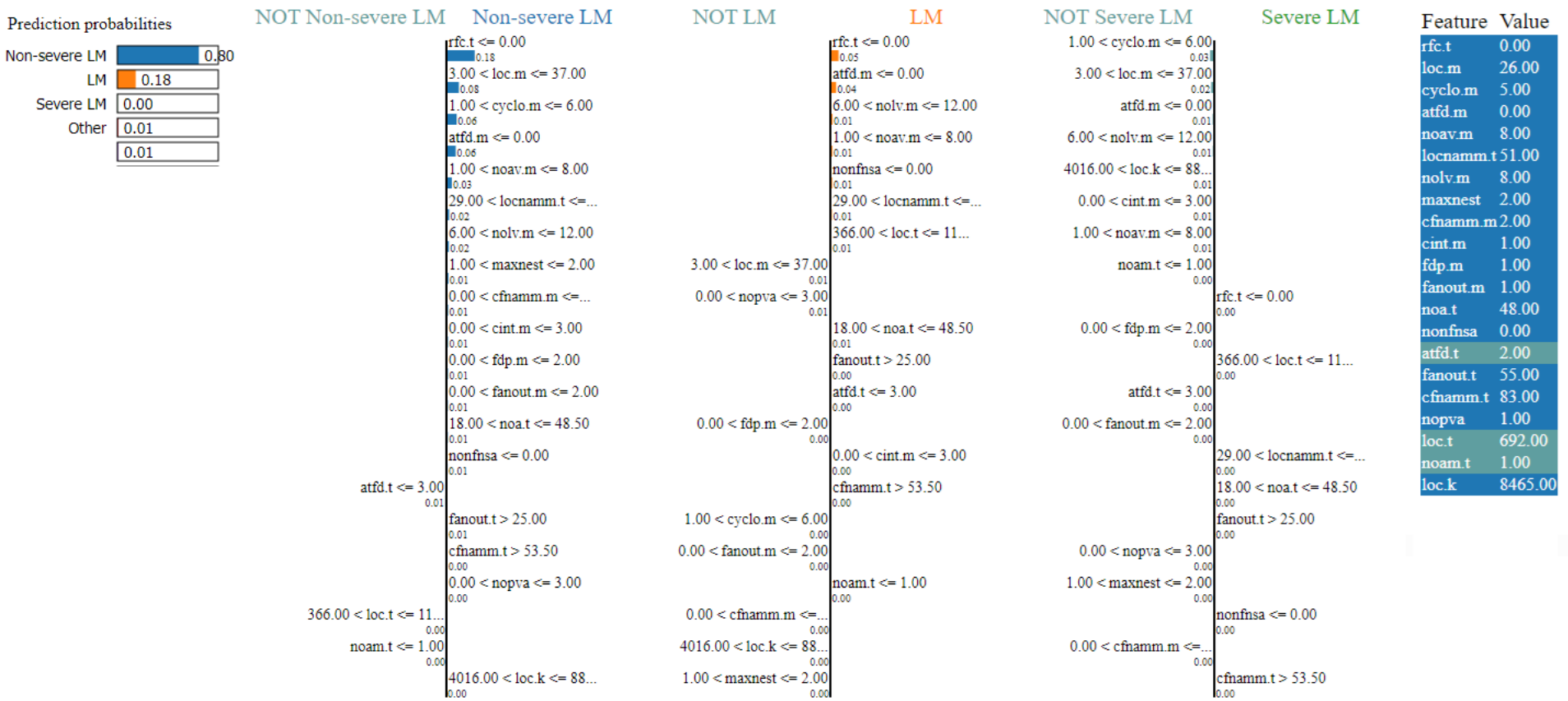

Figura 31 – LIME Explanation for a Non-severe LM Instance in C#

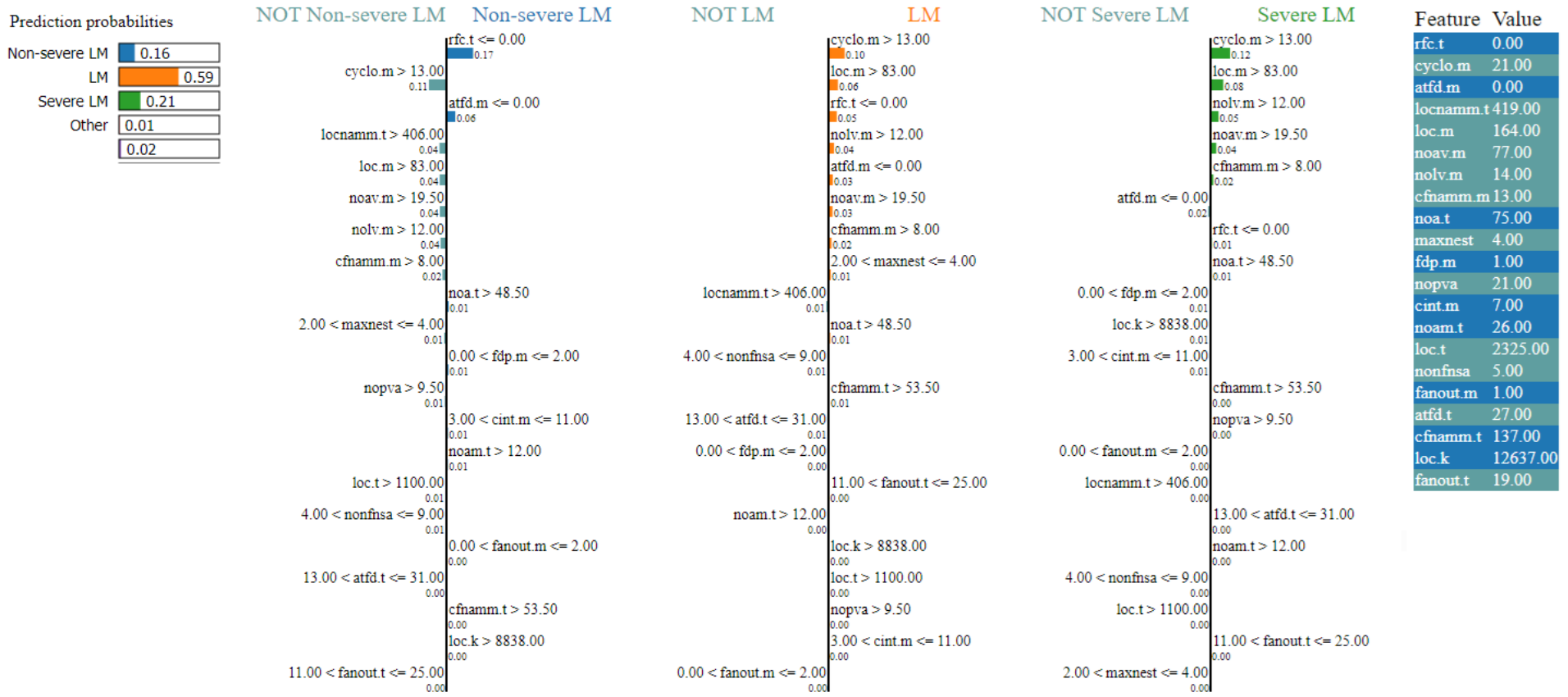

Figura 32 – LIME Explanation for an LM Severity Instance in C#

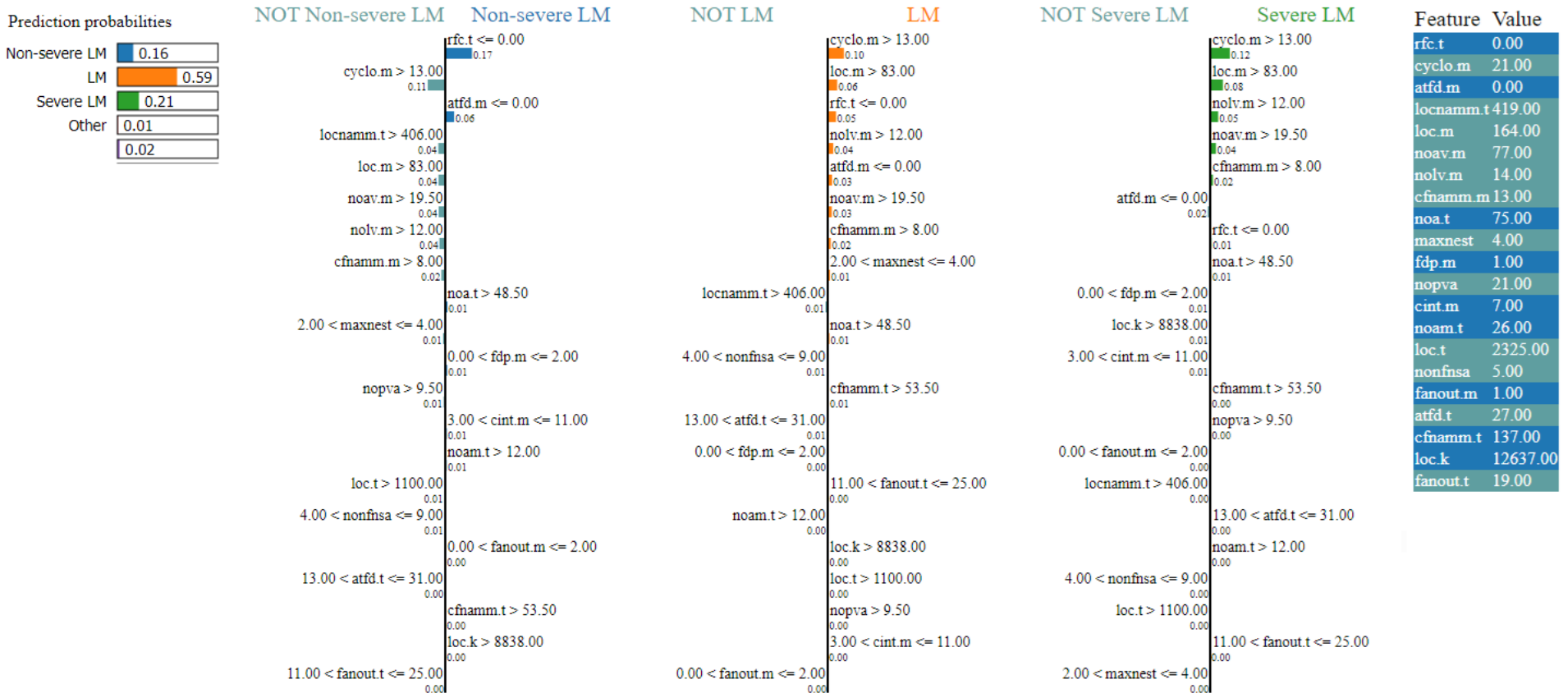

Figura 33 – LIME Explanation for a Severe LM Instance in C#

APPENDIX G – EXPLAINABILITY GRAPHS - DC C#

G.1 Feature Importance

Figura 34 shows the importance of features for detecting DC in C# projects. A total of 21 features were selected using the ANOVA technique, which corresponds to 25% of the available features (84). Feature importance was calculated using the *LossFunctionChange* option in the *get_feature_importance* method of CatBoost. The metrics that stood out the most for detecting DC in C# projects were NOA_type, NOAM_type, and LOC_method.

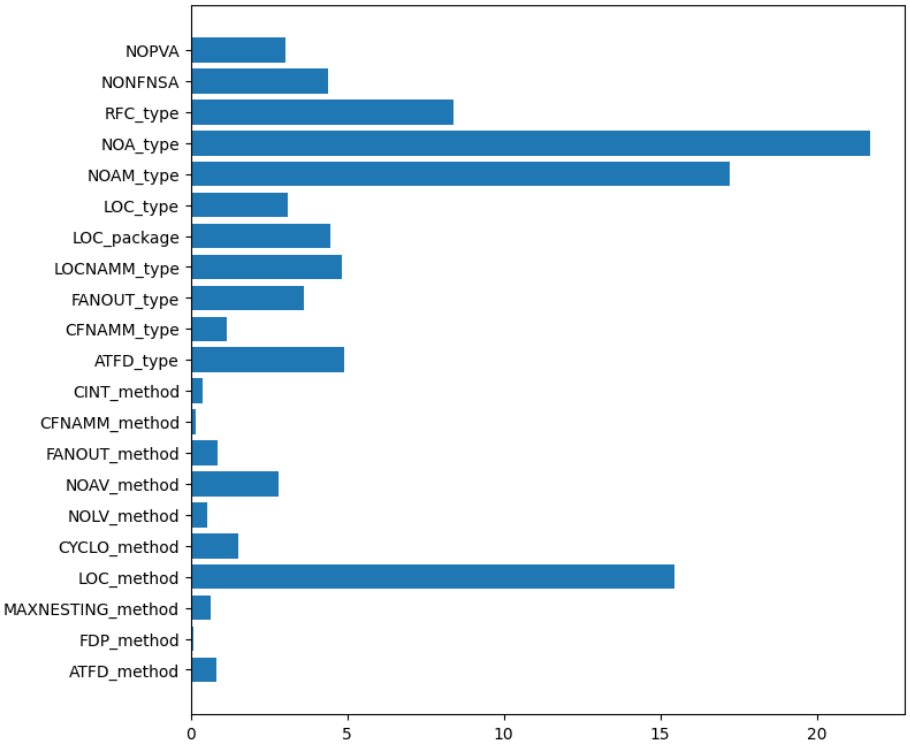

Figura 34 – Feature Importance for DC Detection in C#

A Figura 35 presents the importance of features for severity assessment of DC in C# projects using an RF classifier. Features were selected using the Chi-Squared technique, selecting 25% of the available attributes. Feature importance was computed by aggregating the total reduction in impurity of the *Log Loss* criterion across the trees in the model. LOC_method, NOAV_method, and WMC_type were the most influential attributes in assessing DC severity in C# projects.

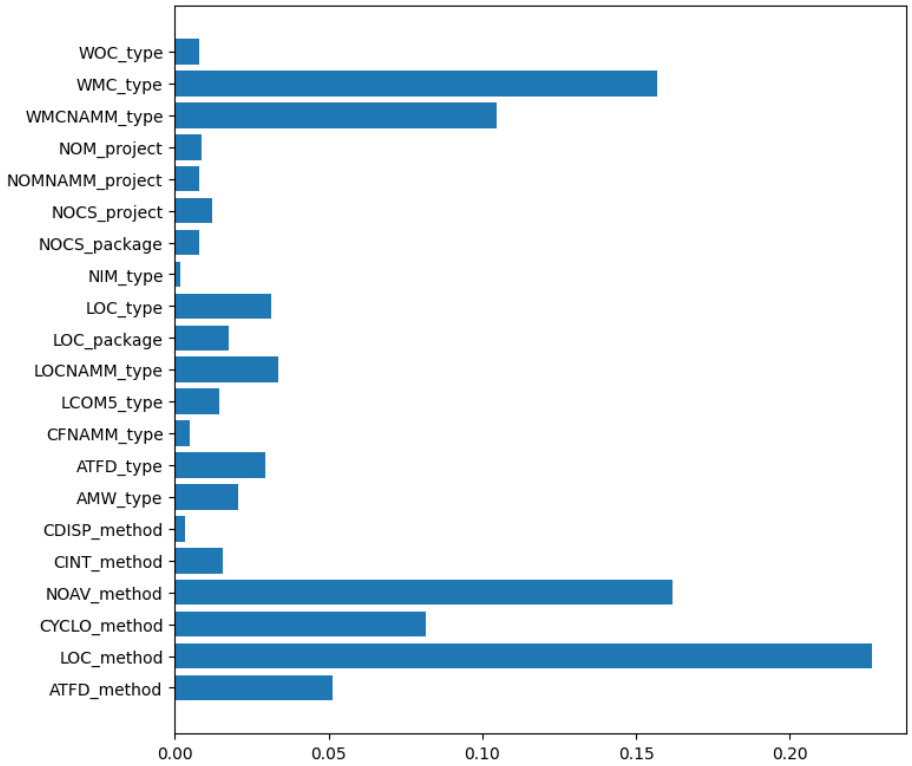

Figura 35 – Feature Importance for DC Severity Assessment in C#

G.2 SHAP

Figura 36 presents the SHAP summary plot showing the impact of each feature on the output of the CatBoost model for DC detection. The features with the greatest impact on the models output were NOAM_type, NOPVA, and NONFNSA. Lower values (in blue) of NOAM_type and NONFNSA negatively impacted the models output, whereas higher values (in red) positively impacted it. In contrast, for NOPVA, higher values had a negative influence, while lower values positively influenced the output of the CatBoost model for detecting DC in C#.

G.3 LIME

For assessing DC severity in C# projects, a model based on Random Forest (RF) was used without scaling the predictor variables. Figure 37 shows the LIME chart for a non-severe DC instance, which had a 74% probability of being correctly classified by the severity assessment model. The most relevant features for this instance were: $WMC\_type \leq 13.00$, $LOC\_method \leq 3.00$, $NOAV\_method \leq 1.00$, $3.00 < WMCNAMM\_type \leq 50.00$, $LCOM5\_type \leq 0.83$, and $LOC\_package \leq 1367.50$. These feature values contributed to the model achieving a high prediction confidence. However, the same featuresexcept for LOC_packagealso contributed (to a lesser extent) to this instance being misclassified as DC (24% probability).

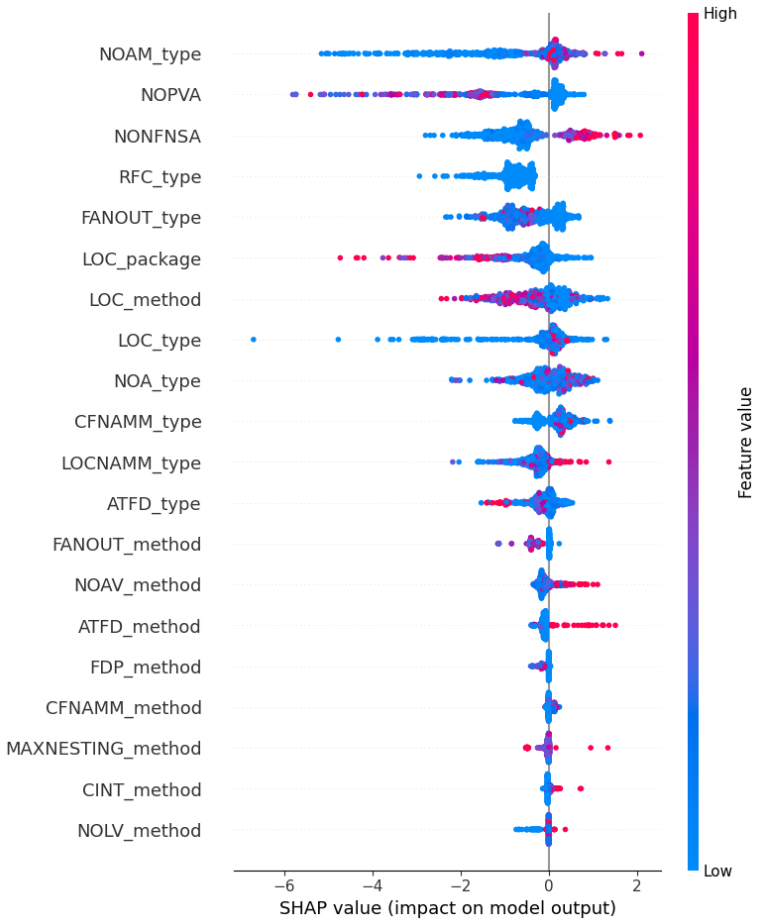

Figura 36 – Impact on the Output of the DC Detection Model in C# - SHAP

For the DC severity instance shown in Figure 38, the correct classification probability was low, at 36%. This caused the model to incorrectly classify the instance as Severe DC (49% probability). Additionally, there was a lower probability (15%) of this instance being predicted as Non-Severe DC. The main reason for this result lies in the positive impact of the WMC_type attribute (> 13.00) on all severity levels, with the greatest contribution observed for the Severe DC level (0.13).

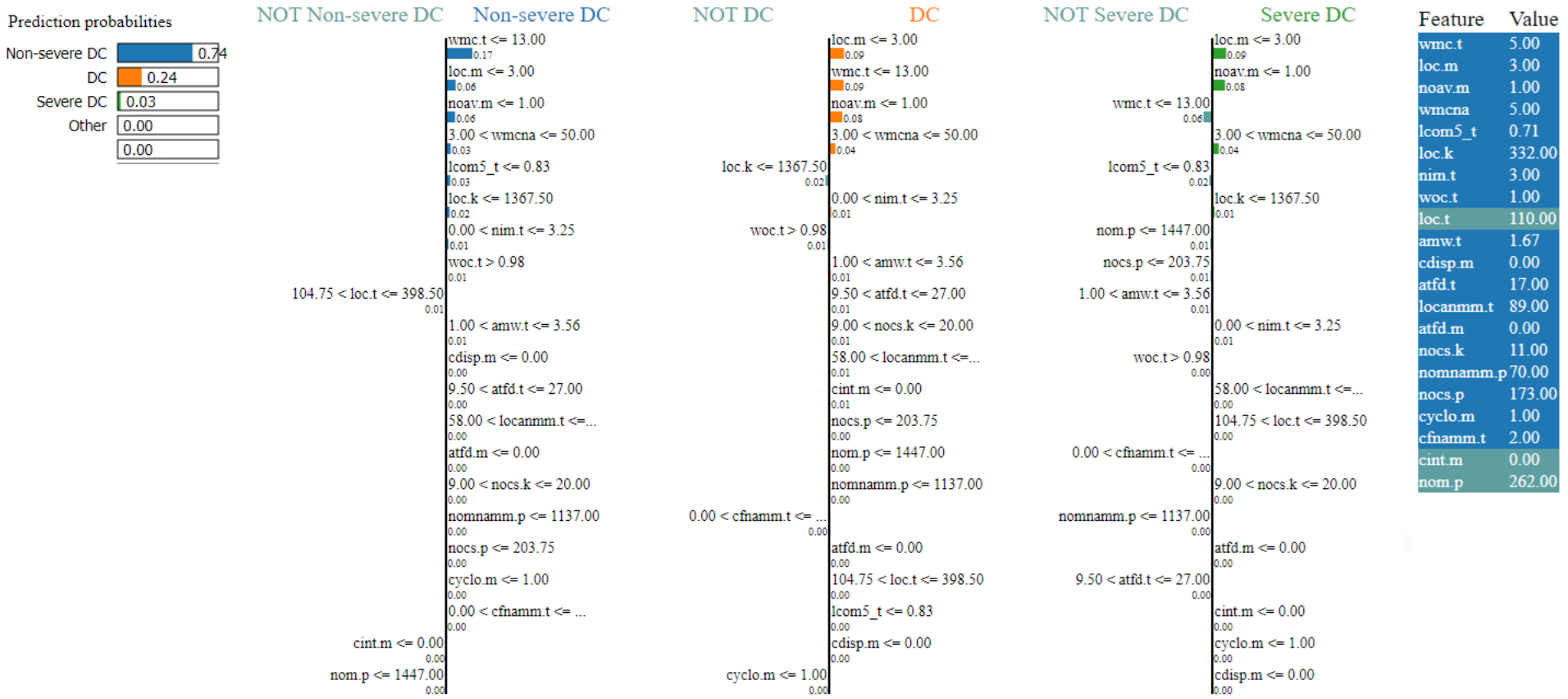

Figura 37 – LIME Explanation for a Non-Severe DC Instance in C#

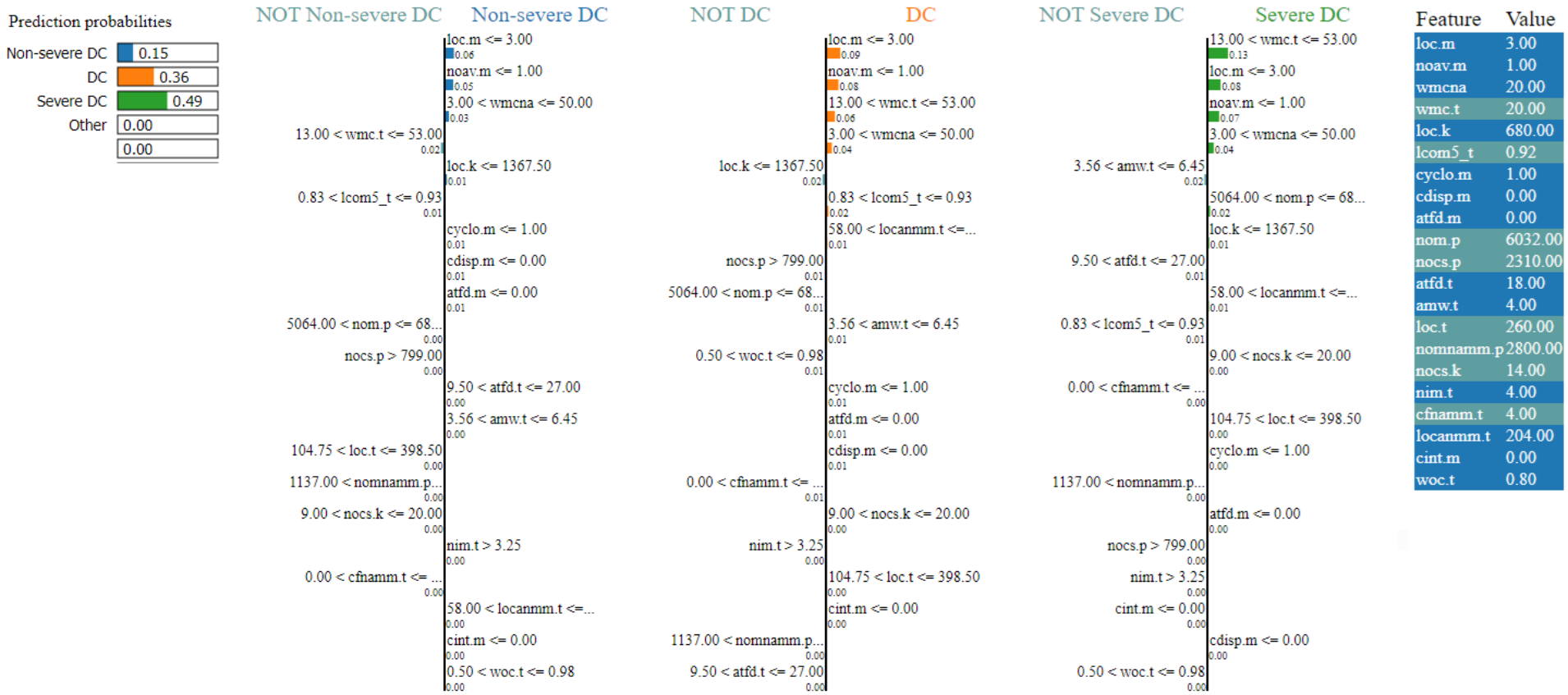

Figura 38 – LIME Explanation for a DC Severity Instance in C#

APPENDIX H – EXPLAINABILITY GRAPHS GC C#

H.1 Feature Importance

A Figura 39 shows the feature importance for detecting GC in C# projects. The *LossFunctionChange* option was used in the *get_feature_importance* method from CatBoost to calculate this importance. Among the 17 features selected using the Chi-Square techniquerepresenting 20% of the original 84 featuresWMCNAMM_type, LOC_package, and LCOM5_type were the most relevant for the detection model.

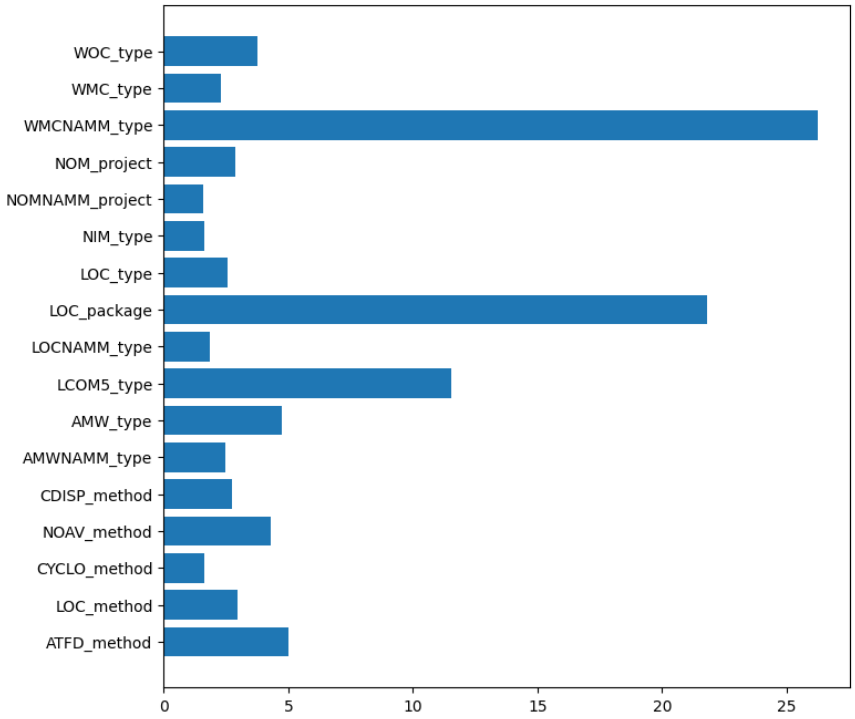

Figura 39 – Feature Importance for GC Detection in C#

Figura 40 presents the feature importance for GC severity assessment in C# projects using an XGBoost classifier. The features were selected using the Chi-Square technique, representing 30% of the available features. Feature importance was computed using XGBoost's default *importance_type* parameter value, *weight*, which measures importance by the number of times a feature appears in the decision tree nodes. As shown in Figure 40, AFTD_method, CYCLO_method, and LOC_method were the top contributors to GC severity prediction in C# projects.

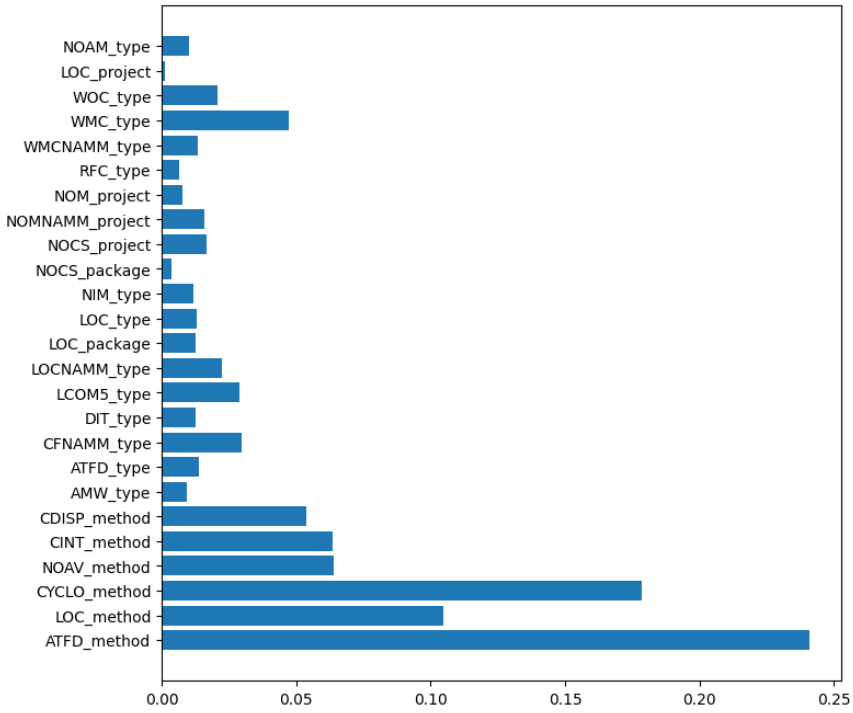

Figura 40 – Feature Importance for GC Severity Assessment in C#

H.2 SHAP

Figura 41 presents the SHAP summary plot showing the impact of features on the CatBoost model's output for GC detection. LOC_package, WMCNAMM_type, LCOM5_type, and AMW_type were the most impactful features. These features share a common behavior: their lower values (blue) negatively affect the model output, while their higher values (red) positively influence the model output for detecting GC in C#.

H.3 LIME

For assessing GC severity in C# projects, an XGBoost-based model was used with standardized predictor data. Figure 42 shows the LIME chart for a Non-severe GC instance, which had a 100% probability of being correctly classified by the severity assessment model. The most relevant features for this instance included: $AMW\_type \leq -0.09$, $NOCS\_package \leq -0.28$, $LCOM5\_type > -0.10$, and $-0.38 < ATFD\_method \leq 0.00$. These feature values contributed to the model achieving maximum prediction probability. Additionally, some feature values such as $NOCS\_project > 0.68$ and $-0.07 < CDISP\_method \leq 0.00$ had the potential to reduce prediction probability but were not strong enough to do so. This happened because AMW_type, NOCS_package, and LCOM5_type exerted strong influence on the XGBoost model, contributing positively by 21%, 16%, and 15%, respectively.

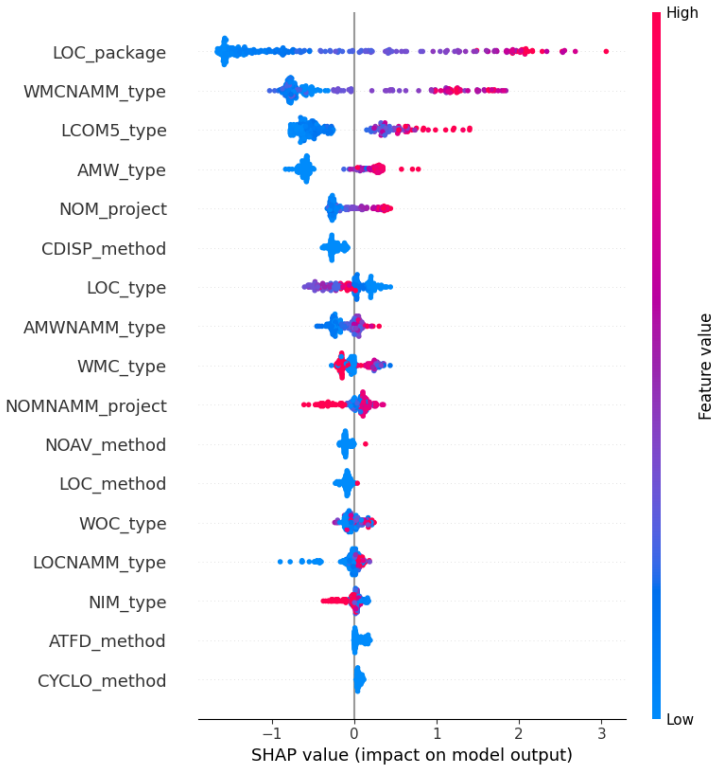

Figura 41 – Impact of Features on GC Detection Model Output in C# SHAP

In the case of a GC severity instance (Figure 43), the prediction probability was also 100%. Most of the feature values contributed to this maximum prediction probability, such as: $LOC\_type > 0.75$, $-0.45 < NOAV\_method \leq -0.04$, $-0.16 < NOAM\_type \leq 0.00$, $NOMNAMM\_project > 0.39$, $0.10 < WMC\_type \leq 0.71$, $0.02 < LOCNAMM\_type \leq 1.03$, $-0.01 < NIM\_type \leq 0.95$, $LOC\_package \leq -0.45$, and $-0.38 < ATFD\_method \leq 0.00$. Conversely, only a few feature values negatively influenced the prediction probability$-0.27 < WOC_type \leq -0.09$, $NOCS\_package \leq -0.28$, and $AMW_type \leq -0.09$but were not strong enough to affect the models excellent performance for this severity level.

Finally, Figure 44 shows a 100% prediction probability for a Severe GC instance. The following feature values contributed positively to this outcome: $LOC\_type > 0.75$, $ATFD\_type > 0.56$, $CFNAMM\_type > 0.79$, $-0.45 < NOAV\_method \leq -0.04$, $LOCNAMM\_type > 1.03$, and $LCOM5\_type > -0.10$. However, features such as $AMW\_type \leq -0.09$, $-0.09 < WOC\_type \leq 0.17$, and $NOCS\_package \leq -0.28$ had a small but insufficient negative contribution in this severity instance.

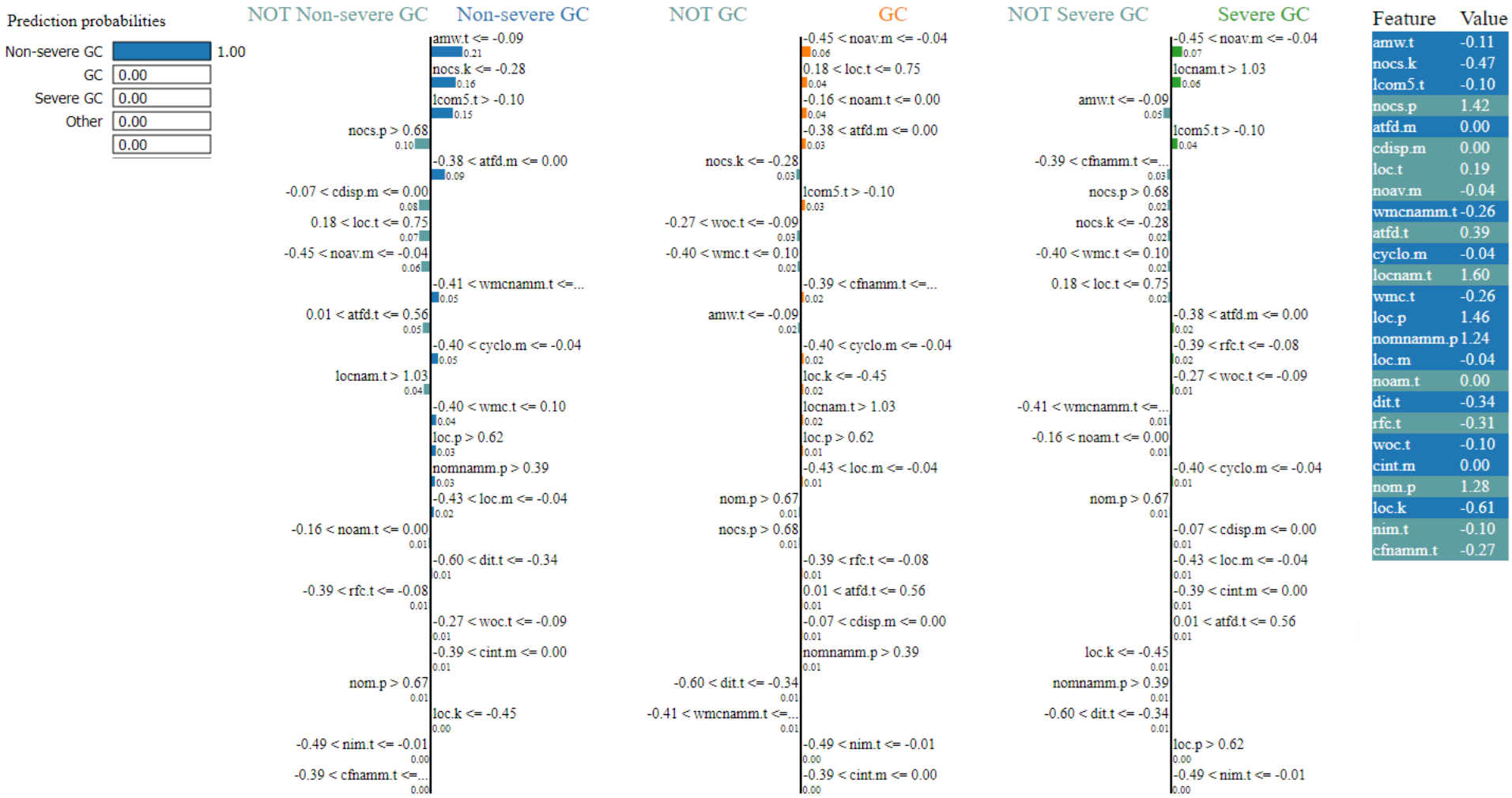

Figura 42 – Explanation of a Non-severe GC Instance in C# using LIME

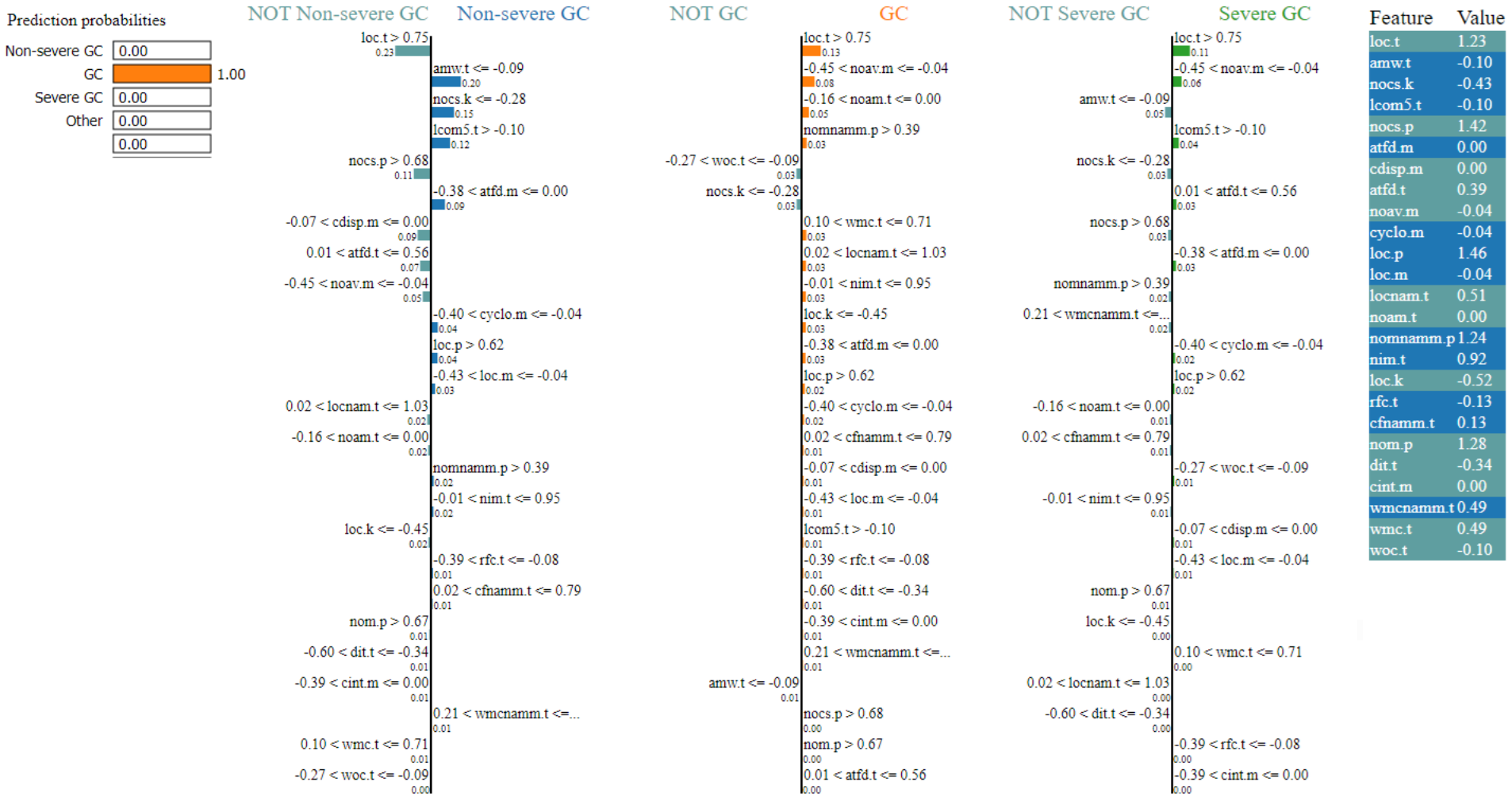

Figura 43 – Explanation of a GC Severity Instance in C# using LIME

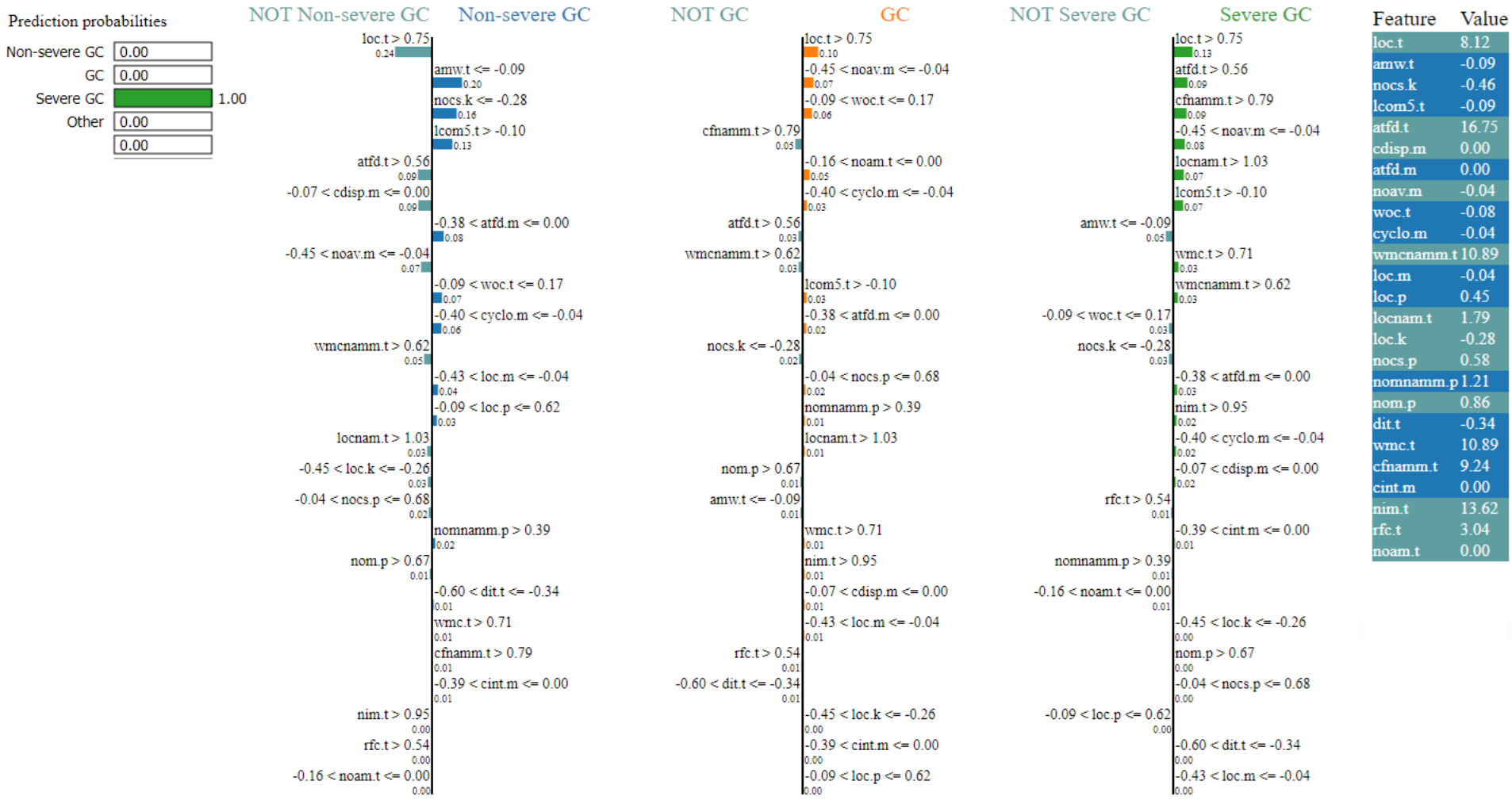

Figura 44 – Explanation of a Severe GC Instance in C# using LIME